

Tema 6 - Elementos de programación funcional

Contenido Teórico

Objetivos

- **Identificar qué nos aporta el estilo funcional** dentro de Python.
- **Usar funciones como datos:** asignarlas a variables, pasarlas como argumentos y devolverlas desde otras funciones.
- **Aplicar transformaciones:** filtros, reducciones mediante comprensiones, `map()`, `filter()` y `reduce()`.
- **Usar funciones lambda** cuando aporten claridad y evitarlas cuando hagan el código menos mantenible.
- **Construir pequeños flujos de tratamiento de datos** orientados a inventario, logs, servicios y métricas.

Qué es programación funcional en Python

Es un estilo de programación que permite organizar el código como transformaciones sobre datos.

Aunque Python no es un lenguaje funcional puro (como Haskell que usa los principios de programación funcional pura), permite usar ideas funcionales de forma práctica:

- **Funciones de orden superior:** pasar funciones como argumentos a otras funciones
- **Funciones Lambda:** Pequeñas funciones anónimas que se definen en una sola línea.
- **Inmutabilidad opcional:** las tuplas representan secuencias que no deberían modificarse.

En automatización de TI resulta útil para procesar datos como listas de servicios, logs, métricas, usuarios o inventarios.

No sustituye a funciones, clases o bucles: los complementa cuando mejora la claridad en la programación.

```
servicios = [" SSH ", " NGINX ", " api "]

normalizados = [s.strip().lower() for s in servicios]
print(normalizados)
```

Funciones como objetos

En Python una función también es un objeto.

- Puede asignarse a una variable
- Guardarse en una colección
- Pasarse como argumento.

Esta capacidad permite construir código flexible: una función puede recibir como dato otra función.

Como veremos más adelante, esto es la base de `map()`, `filter()`, `sorted(key=.)` y de muchas APIs modernas.

```
def normalizar(nombre):
    return nombre.strip().lower()

# operacion se asigna a una función
operacion = normalizar

print(operacion(" SSH "))
print(type(operacion))
```

Funciones pequeñas y reutilizables

El estilo funcional recomienda el uso de funciones con una tarea clara y una salida predecible.

- Por ejemplo, como veremos más adelante, una función de transformación recibe un dato y devuelve otro dato.
- **No debería mezclar tareas sin necesidad:** Por ejemplo, en una sola función hacer cálculo, impresión, lectura de teclado y modificación global.
- Esto facilita probar la función y reutilizarla en scripts, notebooks o módulos.

```
def puerto_valido(puerto):
    return 1 <= puerto <= 65535

def etiquetar_puerto(puerto):
    if puerto_valido(puerto):
        return f"{puerto}: valido"
    return f"{puerto}: fuera de rango"

print(etiquetar_puerto(443))
```

Funciones puras y efectos laterales no deseados

Una función pura:

- No modifica datos externos ni depende de estados ocultos, sólo calcula y devuelve un resultado.
- No debe modificar datos externos ni depender de estados ocultos, sino que debe devolver resultados nuevos.

En Python no siempre será posible escribir funciones puras, pero usar estos principios ayuda a reducir errores.

Sí, por ejemplo, una función modifica una lista externa, conviene hacerlo de forma explícita y documentada.

```
def normalizar(nombre):
    return nombre.strip().lower()

# No modifica la lista original
servicios = [" SSH ", " API "]
limpios = [normalizar(s) for s in servicios]

print(servicios)
print(limpios)
```

Funciones anidadas

Una función puede definirse dentro de otra.

- La función interna queda disponible solo dentro de la función externa y no se puede usar en el ámbito global del código
- Es útil cuando una operación auxiliar no tiene sentido fuera de ese contexto.

Debe usarse con moderación: si la función interna crece, quizá conviene separarla.

```
def resumen_cpu(valores):
    def media(datos):
        return sum(datos) / len(datos)

    promedio = media(valores)
    maximo = max(valores)
    return promedio, maximo

print(resumen_cpu([20, 35, 80]))
```

Funciones de orden superior

Una función de orden superior cumple al menos una de dos condiciones:

- **Recibe otra función como argumento:**
 - Es una idea central del enfoque funcional.
 - `map()`, `filter()`, `sorted()` y `reduce()` son ejemplos habituales que veremos a continuación
- **Devuelve una función como resultado:**
 - La función interna puede recordar valores del contexto donde fue creada.
 - Esta técnica se conoce como **closure**.

```
# Una función recibe otra función como argumento
def aplicar_a_servicios(servicios, funcion):
    resultado = []
    for servicio in servicios:
        resultado.append(funcion(servicio))
    return resultado
print(aplicar_a_servicios([" SSH ", " API "], str.strip))

# Una función Entrega otra función como resultado
def crear_validador(minimo, maximo):
    def validar(valor):
        return minimo <= valor <= maximo
    return validar
puerto_valido = crear_validador(1, 65535)
print(puerto_valido(443))
print(puerto_valido(70000))
```

map(): transformar cada elemento

`map()` recibe dos argumentos:

- Una función
- La colección de datos sobre los que queremos trabajar.

`map()` realiza iteraciones, aplicando la función que pasamos a cada elemento de la colección, produciendo un iterable con nuevos valores.

- La colección original no se modifica.
- La función aplicada debe devolver el nuevo valor en cada iteración.

Debemos tener en cuenta que en muchos casos una comprensión de listas podría ser igual o más clara: Por lo tanto debemos valorar si nos interesa recurrir a `map()` (podría complicar el código y la legibilidad)

```
def normalizar(servicio):
    return servicio.strip().lower()

servicios = [" SSH ", " NGINX ", " Api "]

normalizados = list(map(normalizar, servicios))
print(normalizados)
print(servicios)
```

Iterables y evaluación perezosa (Lazy Evaluation)

Funciones como `map()` y `filter()` que veremos a continuación, no devuelven directamente una lista, sino que devuelven objetos iterables

Esto es el principio de **evaluación perezosa** (Lazy Evaluation):

- Es una de las bases de los lenguajes funcionales. Para mejorar el rendimiento, el cálculo se realiza cuando se necesita.
- Es lo contrario a la evaluación inmediata, donde se calculan todos los valores en el momento.
- Esto significa que los valores de `map()` no los vas a obtener hasta que no los recorras.

Para visualizar todos los resultados de una vez, solemos convertir la salida de estas funciones con `list()`, `tuple()` o `dict()` lo que obliga a hacer el recorrido completo.

- Esto evita trabajo innecesario en colecciones grandes.

```
servicios = ["ssh", "nginx", "api"]

resultado = map(str.upper, servicios)
print(resultado)

print(list(resultado))
# El iterable ya se consumió
print(list(resultado))
```

map() con más de una colección

`map()` también puede recibir más de un iterable.

- La función debe aceptar tantos argumentos como colecciones se pasen.
- El recorrido termina cuando se agota la colección más corta.

Es útil para combinar datos paralelos, aunque debe mantenerse legible.

```
def crear_endpoint(servicio, puerto):
    return f"{servicio}:{puerto}"

servicios = ["ssh", "nginx", "postgresql"]
puertos = [22, 443, 5432]

endpoints = list(map(crear_endpoint, servicios, puertos))
print(endpoints)
```

filter(): seleccionar elementos

`filter()` recibe dos argumentos:

- **Una función:** que debe evaluar `True` o `False` en base a lo que vea en el iterable de la colección recibida.
- **Una colección:** con elementos que vamos a evaluar

`filter()` conserva los elementos para los que la función devuelve `True` y descarta los elementos para los que devuelve `False`.

- Esa función suele llamarse **predicado**.
- La colección original no se modifica.

Es útil para separar servicios activos, errores de log, puertos no permitidos o usuarios bloqueados entre otros.

```
def esta_activo(servicio):
    return servicio["activo"]

servicios = [
    {"nombre": "ssh", "activo": True},
    {"nombre": "api", "activo": False},
    {"nombre": "http", "activo": False},
    {"nombre": "ntp", "activo": True},
]

activos = list(filter(esta_activo, servicios))
print(activos)
```

filter() con diccionarios

Para filtrar un diccionario se suele trabajar con `items()`.

- Cada elemento recibido es una tupla con clave y valor.
- Después puede reconstruirse el diccionario con `dict()`.

Esta técnica es útil para limpiar configuraciones o reportes estructurados.

```
puertos = {
    "ssh": 22,
    "nginx": 443,
    "api": 8080,
}

def es_estandar(item):
    servicio, puerto = item
    return puerto in (22, 80, 443)

estandar = dict(filter(es_estandar, puertos.items()))
print(estandar)
```

Comprensiones frente a map() y filter()

- Las comprensiones suelen ser la opción más propia (o **Pythonic** como se suele usar en argot de programación Python) para transformar o filtrar listas.
- `map()` y `filter()` son útiles cuando ya tenemos funciones reutilizables.
- **No hay una única regla universal:** manda la legibilidad.
- En equipos de trabajo generando código, conviene elegir la forma que se entienda antes por todos

```
servicios = [" SSH ", " Api ", " NGINX "]

# Comprensión clara
limpios1 = [s.strip().lower() for s in servicios]

# map poco claro. Dos anidaciones
limpios2 = list(map(str.lower, map(str.strip, servicios)))

print(limpios1)
print(limpios2)
```

lambda: funciones anónimas

- Siguiendo los principios de la programación funcional, la mayoría de las veces, las funciones que usamos suelen ser muy pequeñas.
- `lambda` permite crear funciones pequeñas sin nombre, de forma anónima, e incrustarlas directamente en llamadas a `map()`, `filter()` o `reduce()`.
- **Sintaxis:**

```
lambda parametros: expresión
```

```
servicios = ["ssh", "nginx", "postgresql", "api"]

longitudes = list(map(lambda s: len(s), servicios))
print(longitudes)

activos = filter(lambda s: s != "api", servicios)
print(list(activos))
```

Usar lambda con criterio

- Se recomienda usar `lambda` solo cuando la expresión cabe en una línea y se entiende de inmediato.
- No debe usarse para ocultar lógica compleja.
- `lambda` mejora el código cuando evita crear una función trivial, de esta forma evitamos incluso ponerle nombre
- Pero puede empeorar la lectura si contiene condiciones largas o transformaciones difíciles.
- La claridad suele ser más importante que ahorrar líneas.
- **Una regla simple:** si hay que explicar demasiado la función o la operación necesita varias instrucciones, usa `def`.

```
# Aceptable: expresión breve
puertos = [22, 443, 8080]

print(list(map(lambda p: p + 1, puertos)))

# Más claro con def si la función crece
def describir_puerto(p):
    return f"puerto {p}"

print(list(map(describir_puerto, puertos)))
```

reduce(): reducir a un solo resultado

`reduce()` aplica una función acumuladora a una colección.

- Devuelve un único resultado final.
- Debe importarse desde `functools`.
- **En cada iteración, la función recibe dos valores:** El acumulador parcial y el siguiente elemento de la colección.
- El resultado devuelto pasa a ser el nuevo acumulador para la siguiente iteración.

Para sumas simples, `sum()` suele ser más claro. `reduce()` se reserva para acumulaciones menos directas.

```
from functools import reduce
medidas = [12, 18, 7, 20]

def acumular(total, valor):
    return total + valor

print(reduce(acumular, medidas))

# Con sum obtenemos el mismo resultado
print(sum(medidas))

# Retornar una cadena con todos los
# nombres en mayúsculas separados por " : "
nombres = ["ssh", "rdp", "www", "ntp"]

def concatenar(acumulador, nombre):
    return acumulador.upper()+" : "+nombre.upper()

print(reduce(concatenar, nombres))
```

reduce() con valor inicial

`reduce()` puede recibir un valor inicial.

- Esto evita problemas con colecciones vacías y define explícitamente el acumulador inicial.
- Es más seguro cuando el origen de datos puede no traer elementos, de esa forma evitamos error.

En scripts reales es mejor preferir la versión más fácil de leer.

```
from functools import reduce

puertos = [8080, 22, 8443, 443, 1024]

# La función lambda tiene dos parametros:
# acc (acumulador) y p (iterable)
# Si el elemento actual es mayor que el acumulador
# se queda con ese elemento
# el primer valor es 0
# al final, el mayor valor está en el acumulador.
maximo = reduce(lambda acc, p: p if p > acc else acc, puertos, 0)

print(maximo)
```

sorted(key=..): ordenar con una función

`sorted()` puede recibir una función mediante `key`.

- Esa función calcula el criterio de ordenación para cada elemento de la colección iterable
- No modifica la colección original.

```
servicios = [
    {"nombre": "ssh", "puerto": 22},
    {"nombre": "api", "puerto": 8080},
    {"nombre": "nginx", "puerto": 443},
    {"nombre": "rdp", "puerto": 3389},
]

# Recuerda la regla de orden de argumentos:
# Primero argumentos posicionales
# Después argumentos con nombre
# De ahí que en sorted va primero la colección
# y después la función de ordenado
ordenados = sorted(servicios, key=lambda s: s["puerto"])
print(ordenados)
```

any() y all()

`any()` devuelve `True` si algún elemento del iterable evalúa como verdadero.

`all()` devuelve `True` si todos los elementos del iterable evalúan como verdaderos.

- No reciben una función como `map()` o `filter()`, sino que reciben un iterable de valores booleanos, o evaluables como booleanos.
- Son útiles para validar colecciones sin escribir bucles largos.


```
puertos = [22, 443, 8080]

hay_puerto_alto = any(p > 1024 for p in puertos)
todos_validos = all(1 <= p <= 65535 for p in puertos)

print(hay_puerto_alto)
print(todos_validos)
```

Expresiones generadoras

Una **expresión generadora** se parece a una comprensión, pero **no crea la colección completa de golpe, sino que produce elementos bajo demanda**.

- Es útil cuando solo necesitamos recorrer los resultados una vez.

```
logs = ["OK ssh", "ERROR api", "ERROR nginx"]

# Expresión generadora asignada a errores
errores = (linea for linea in logs
           if linea.startswith("ERROR"))

print(errores) # Generator object
# errores produce las lineas
# que empiezan por ERROR
for error in errores:
    print(error)
```

Pipelines simples de datos

Un pipeline funcional encadena pasos: limpiar, filtrar, transformar y resumir.

- Cada paso debe ser comprensible de forma independiente.
- Es mejor dividir el flujo en variables intermedias si eso ayuda a depurar.

Este enfoque encaja con procesamiento de logs, inventarios y respuestas de APIs.

```
logs = [" ERROR ssh ", " OK nginx ", " ERROR api "]

limpios = [line.strip() for line in logs]
print(limpios)

errores = [line for line in limpios if line.startswith("ERROR")]
print(errores)

servicios = [line.split()[1] for line in errores]
print(servicios)
```

Aplicar funciones a objetos

Las técnicas funcionales también pueden operar sobre objetos.

- Es habitual extraer atributos, filtrar por estado o construir reportes.

- **No hace falta que el objeto sea complejo:** basta con que exponga datos o métodos claros.
- Esta idea conecta clases, colecciones y funciones.

```
class Servicio:
    def __init__(self, nombre, puerto):
        self.nombre = nombre
        self.puerto = puerto

    def es_web(self):
        return self.puerto in (80, 443)

# Creamos una lista de objetos
red = [
    Servicio("ssh", 22),
    Servicio("http", 80),
    Servicio("rdp", 3389),
    Servicio("https", 443)
]

# Aplicamos el método 'es_web' a cada objeto de la lista
servicios_web = filter(lambda s: s.es_web(), red)

print([s.nombre for s in servicios_web])  # ['http', 'https']
```

Transformar sin modificar el original

Como hemos citado anteriormente, en un enfoque funcional se recomienda crear nuevos resultados en lugar de modificar la entrada.

- Esto reduce efectos laterales y facilita comparar antes/después.
- En Python hay casos donde modificar es correcto, pero debe ser una decisión explícita.

En automatización de TI, conservar los datos originales ayuda a diagnosticar errores.

```
original = [" SSH ", " Api "]

normalizados = [s.strip().lower()
                 for s in original]

print("original:", original)
print("nuevo:", normalizados)
```

Buenas Prácticas y Errores Frecuentes

Buenas Prácticas:

- Usar funciones pequeñas y con nombres claros.
- Preferir comprensiones cuando expresen la intención con claridad.
- Usar lambda solo para expresiones breves.
- Convertir `map()` y `filter()` a `list()` o `dict()` cuando se quiera materializar el resultado.
- Evitar pipelines demasiado compactos si dificultan la depuración.

Errores Frecuentes:

- Pensar que `map()` y `filter()` devuelven listas directamente.

- Consumir un iterador y volver a intentar leerlo como si siguiera completo.
- Usar lambda para lógica que debería ser una función con nombre.
- Usar `reduce()` cuando `sum()`, `min()`, `max()` o un bucle son más claros.
- Modificar objetos dentro de `map()` sin hacerlo explícito.

```
def es_puerto_web(puerto):
    return puerto in (80, 443, 8080)

puertos = [22, 80, 443, 5432]
web = [p for p in puertos if es_puerto_web(p)]

print(web)
```

Resumen del Tema 6

Python permite combinar estilos:

- imperativo
- orientado a objetos
- funcional

Las funciones pueden tratarse como datos y pasarse a otras funciones:

- `map()` transforma
- `filter()` selecciona
- `reduce()` acumula.

lambda es útil para expresiones breves, no para ocultar lógica compleja.

La prioridad en el uso en código profesional sigue siendo la **claridad**, la **trazabilidad** y el **mantenimiento**.

```
def resumen_funcional(operacion):
    tipos = {
        "map": "transformar",
        "filter": "seleccionar",
        "reduce": "acumular",
        "lambda": "función anónima",
    }
    return tipos.get(operacion, "revisar")

print(resumen_funcional("filter"))
```

Flujo de trabajo en el laboratorio

- Desde el repositorio Git podrás cargar el directorio del Tema06.
- Primero se revisarán transformaciones y filtros con funciones normales.
- Después se aplicarán lambda, `map()`, `filter()`, `reduce()`, `any()`, `all()` y expresiones generadoras.
- La versión de Python usada en el venv del curso será Python 3.13.

```
# Cargar el entorno
cd ~/Curso_Python
source .venv/bin/activate
python --version

# Actualizar del repositorio Git
git pull origin main

# Para ejecutar VSCode
code . # usar carpeta Tema06

# Para ejecutar en Terminal
python
```